

PLATAFORMA DE TELEMETRIA LOGÍSTICA

NexoFrota

Persistência poliglota, geolocalização e análise operacional em uma arquitetura reproduzível.

PostgreSQL / MongoDB / Streamlit / GeoJSON

Responsável: Lucas Gabriel

1. Resumo executivo

A NexoFrota é uma plataforma para acompanhamento operacional de veículos de carga. O sistema mantém cadastros relacionais, recebe eventos de telemetria, materializa a posição atual, executa consultas geoespaciais e apresenta os resultados em um painel web.

A solução demonstra persistência poliglota: PostgreSQL protege relações e restrições cadastrais; MongoDB armazena eventos em GeoJSON e atende buscas por proximidade. A camada de serviços combina as fontes sem acoplar a interface aos detalhes de infraestrutura.

Resultado entregue

Aplicação Streamlit, mapa Folium, gráficos Plotly, ambiente Docker, seed controlado, simulador de movimento, oito testes unitários e documentação operacional.

Objetivos

- Escolher o banco conforme a natureza de cada dado.
- Preservar o histórico sem confundir posições antigas com a posição atual.
- Localizar veículos em um raio e ordenar os mais próximos.
- Unificar dados de fontes distintas por meio de uma camada de serviços.
- Fornecer execução reproduzível para avaliação e desenvolvimento.

Sumário

1. Resumo executivo
2. Requisitos
3. Arquitetura
4. Modelagem de dados
5. Geoespacial
6. Interface e simulação
7. Implantação
8. Segurança e qualidade
9. Testes
10. Limitações e evolução

2. Requisitos e escopo

O escopo cobre o ciclo completo de uma demonstração de telemetria: preparação dos bancos, dados iniciais, leitura operacional, simulação de eventos e visualização.

ID	Requisito	Implementação
RF-01	Manter motoristas e veículos	PostgreSQL com chaves e restrições
RF-02	Registrar telemetria	Coleção MongoDB telemetry
RF-03	Consultar posição atual	Coleção current_positions
RF-04	Buscar por raio	Pipeline \$geoNear e índice 2dsphere
RF-05	Unificar dados	FleetService associa por vehicle_id
RF-06	Exibir mapa	Folium integrado ao Streamlit
RF-07	Exibir indicadores	Métricas, tabelas e Plotly
RF-08	Simular movimento	MovementSimulator validado
RF-09	Consultar histórico	Filtro por veículo e intervalo

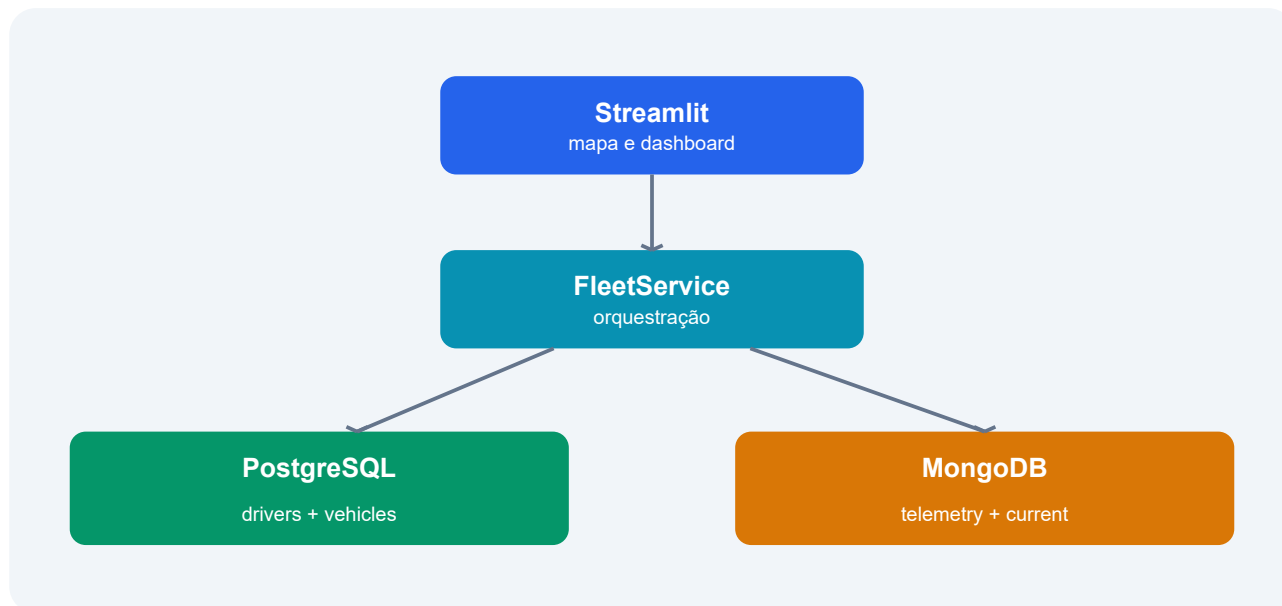
Fora do escopo

Autenticação de usuários, ingestão por dispositivos reais, mensageria, alta disponibilidade e observabilidade distribuída são evoluções para produção.

Critérios de aceite

- Os três serviços iniciam com configuração documentada.
- O seed cria três veículos e 36 leituras sem segredos no código.
- A consulta geoespacial retorna distância em quilômetros.
- Uma simulação acrescenta histórico e atualiza a posição atual.
- O dashboard sinaliza velocidade, parada e temperatura crítica.
- Lint, compilação e testes unitários terminam sem erros.

3. Arquitetura da solução



A interface acessa somente a camada de serviços. Os repositórios encapsulam consultas e comandos específicos de cada banco. Modelos de domínio validam coordenadas, datas e velocidade antes da persistência.

Fluxo de leitura



- O PostgreSQL fornece cadastro e vínculo entre motorista e veículo.
- O MongoDB fornece a última leitura de cada veículo.
- O serviço une os resultados em memória por `vehicle_id`.
- A interface calcula visualizações sem executar SQL ou pipelines diretamente.

Fluxo de escrita



A mesma leitura é inserida como evento imutável e aplicada à projeção atual. Em cargas com várias leituras do mesmo veículo, apenas a mais recente atualiza a projeção.

4. Modelagem de dados

Modelo relacional

Tabela	Campo	Tipo e regra
drivers	id	Inteiro, chave primária
drivers	name	Varchar(120), obrigatório
drivers	license_number	Varchar(20), único
drivers	status	active, inactive ou leave
vehicles	id	Inteiro, chave primária
vehicles	plate	Varchar(10), único
vehicles	model	Varchar(100), obrigatório
vehicles	driver_id	Chave estrangeira única
vehicles	active	Booleano, padrão true

Documento de telemetria

```
{
  "vehicle_id": 101,
  "location": {
    "type": "Point",
    "coordinates": [-46.6333, -23.5505]
  },
  "speed_kmh": 56.0,
  "cargo_temperature_c": 4.0,
  "recorded_at": "2026-09-27T12:00:00Z"
}
```

Regra GeoJSON

A ordem obrigatória é longitude, latitude. Datas são armazenadas como BSON Date em UTC para preservar ordenação e comparação cronológica.

5. Histórico, projeção e geoespacial

A coleção telemetry é um log de eventos: nenhuma leitura anterior é substituída. A coleção current_positions contém um documento por veículo e representa o estado atual.

Coleção	Índice	Finalidade
telemetry	location 2dsphere	Consultas espaciais sobre eventos
telemetry	vehicle_id + recorded_at	Histórico ordenado por veículo
current_positions	location 2dsphere	Raio e proximidade atuais
current_positions	recorded_at desc	Recência operacional

Consulta por raio

```
{
  "$geoNear": {
    "near": {"type": "Point", "coordinates": [longitude, latitude]},
    "distanceField": "distance_meters",
    "maxDistance": radius_km * 1000,
    "spherical": true
  }
}
```

Por que não consultar apenas o histórico?

Uma leitura antiga pode estar dentro do raio enquanto a posição atual está fora. Consultar a projeção evita esse falso positivo e reduz o volume examinado.

Consistência

A escrita ocorre primeiro no histórico e depois na projeção. O desenho favorece auditoria: se a segunda etapa falhar, a posição atual pode ser reconstruída a partir do evento mais recente de cada veículo.

6. Interface, indicadores e simulação

O painel organiza a operação em quatro abas. O mapa aplica o ponto e o raio definidos na barra lateral; as demais abas apresentam estado atual e evolução temporal.

Área	Conteúdo
Mapa	Veículos no raio, marcadores, círculo e lista dos mais próximos
Visão geral	Cadastro, leitura atual, coordenadas e alertas
Dashboard	KPIs, velocidade e temperatura por veículo
Histórico	Séries por veículo e janela de tempo

Regras de alerta

Condição	Regra	Sinalização
Velocidade	Acima de 80 km/h	Excesso de velocidade
Parada	Abaixo de 1 km/h	Veículo parado
Temperatura	Fora de -20 a 8 °C	Temperatura crítica
Ausência	Sem posição atual	Sem telemetria

Simulação

O MovementSimulator recebe a última leitura, escolhe ângulo e distância dentro do limite configurado, ajusta coordenadas e varia velocidade e temperatura. Latitude, longitude e velocidade são limitadas a domínios válidos.

Testabilidade

A fonte aleatória pode ser injetada. Os testes usam uma semente fixa, tornando o deslocamento repetível sem alterar o comportamento da aplicação.

7. Implantação e operação

O Docker Compose define três serviços com verificações de saúde e volumes persistentes. A aplicação só inicia após PostgreSQL e MongoDB aceitarem conexões.

Serviço	Porta do host	Persistência
app	8501	Sem estado local
postgres	5433	Volume postgres_data
mongo	27018	Volume mongo_data

Inicialização

```
Copy-Item .env.example .env
docker compose up -d --build
docker compose exec app python -m nexofrota.seed --if-empty
```

Operação local

```
docker compose up -d postgres mongo
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
$env:PYTHONPATH = "src"
python -m nexofrota.seed --if-empty
streamlit run src/app.py
```

Operação destrutiva

O modo --reset remove os cadastros e as duas coleções antes de recriar a amostra. Deve ser usado somente em desenvolvimento.

8. Segurança e qualidade

Controles implementados

- Segredos lidos de variáveis de ambiente; .env fora do versionamento.
- Container da aplicação executado por usuário sem privilégios.
- SQL parametrizado e esquema com restrições de integridade.
- Coordenadas, velocidade e data validadas no domínio.
- Conteúdo textual escapado antes de entrar nos pop-ups do mapa.
- Timeout de conexão e verificação explícita dos dois bancos.
- Seed destrutivo condicionado a uma opção clara de linha de comando.

Aspecto	Prática
Separação	Interface, serviço, repositório, modelo e configuração
Idempotência	Schema, índices, upserts e seed --if-empty
Observabilidade	Logs de inicialização e falhas de seed
Manutenção	Tipos, funções pequenas e decisões registradas em ADRs
Portabilidade	Docker e dependências com faixas de versão

Credenciais de exemplo

Os valores do arquivo .env.example são exclusivos para desenvolvimento local. Ambientes compartilhados exigem senhas fortes e um gerenciador de segredos.

9. Validação e testes

A validação automatizada cobre comportamento de domínio e qualidade estática. A execução registrada durante a entrega concluiu oito testes unitários com sucesso.

Grupo	Verificação
GeoJSON	Conversão do modelo para documento e retorno ao domínio
Validação	Rejeição de latitude fora do intervalo
Alertas	Normal, excesso de velocidade, parada e temperatura
Simulação	Limite de deslocamento e preservação da frota
Seed	36 leituras, três veículos e estado final conhecido
Estático	Ruff, formatação e compilação dos módulos

```
pytest -q
..... [100%]
8 passed

ruff check .
All checks passed!
```

Testes de integração recomendados

- Aplicar o schema em um PostgreSQL limpo e verificar restrições.
- Confirmar índices 2dsphere e composto após o seed.
- Comparar distâncias do pipeline geoespacial com valores conhecidos.
- Executar simulações concorrentes e validar histórico e projeção.

10. Limitações e evolução

A implementação atende demonstração e avaliação acadêmica. Para produção, o volume, a frequência dos eventos e os requisitos de disponibilidade exigem componentes adicionais.

Horizonte	Evolução
Curto prazo	Autenticação, perfis, filtros configuráveis e testes de integração
Médio prazo	Fila de eventos, API de ingestão, métricas e alertas externos
Longo prazo	Particionamento, retenção, replicação e reconstrução por eventos

Recomendações

- Parametrizar limites por carga, contrato e rota.
- Manter `current_positions` por consumidor de eventos ou change stream.
- Adicionar trilha de auditoria de acesso e alterações cadastrais.
- Definir retenção, backups testados e objetivos de recuperação.
- Instrumentar latência de ingestão, atraso de posição e falhas de banco.

Conclusão

A NexoFrota usa cada banco por sua vantagem real: integridade relacional para cadastros e documentos geoespaciais para telemetria. A projeção de posição atual, a camada de serviços e a automação de qualidade formam uma base organizada para evolução.

Artefatos

Código-fonte: desafio mongodb aula4/src

Testes: desafio mongodb aula4/tests

Decisões: desafio mongodb aula4/docs/DECISOES_ARQUITETURA.md

Relatório editável: desafio mongodb aula4/docs/RELATORIO_TECNICO.md